SwayZone Command Center: Project Exo Operator Manual - NotebookLM

Exported on: 5/13/2026, 6:26:28 PM

what should i put here?

Operator, I have analyzed your latest screenshot. You are currently in the **MCP Toolkit** extension within Docker Desktop, looking at the configuration screen for the **MCP Database Server**.

The interface indicates that **"CONFIGURATION REQUIRED"** because your agent needs to know exactly how to connect to your data storage. The specific text box you are looking at is asking for the **database-server.database_url**.

What you put in that box depends entirely on which database engine you have deployed for your Watchtower C2 Dashboard. The screenshot provides the specific syntax you must use for the three supported engines:

- **For SQLite:** You will need to input a string like `sqlite+aiosqlite:///data/mydb.db`. *Tactical Note: Because we previously configured your system to run off your portable D:\Watchtower_Ops drive via a live volume mount, SQLite is a highly probable choice for your local, isolated container architecture. You just need to ensure the /data/mydb.db path accurately reflects where your database file is mounted inside the Linux container.*
- **For PostgreSQL:** You will need to input `postgresql+asyncpg://user:password@localhost:5432/mydb`. (You must replace `user`, `password`, and `mydb` with your actual credentials).
- **For MySQL:** You will need to input `mysql+aiomysql://user:password@localhost:3306/mydb`.

Socratic Check: To ensure we don't cause a connection error anomaly, which specific database engine (SQLite, Postgres, or MySQL) are you currently using to store your scraped OSINT data and agent logs? Tell me your engine, and we can construct the exact connection string you need to paste into that field.

How do I check which database my app is using?

Operator, excellent question. To identify the underlying database architecture of your application, we need to inspect the configuration files and code currently housed in your `D:\Watchtower_Ops` airlock.

While the provided screenshot only shows the three supported database types by the MCP Server (SQLite, PostgreSQL, and MySQL) and their corresponding connection string formats, I can draw on standard external developer knowledge—which you may want to independently verify—to guide your investigation.

Here are the **three tactical locations** to check inside your D: drive workspace:

- **1. The docker-compose.yml File (The Deployment Blueprint):** Open this file and review your defined services. If you see a specific service block pulling a `postgres` or `mysql` image alongside your app, you are running a dedicated database server. If your only service is the Python `app` container we just rebuilt, you are almost certainly using a local **SQLite** database.
- **2. The .env Configuration File (The Environment Secrets):** Check your project folder for a file simply named `.env`. This is where sensitive configuration details are typically kept. Look for a variable named something like `DATABASE_URL` or `DB_CONNECTION`. The prefix of the URL (e.g., `sqlite:///`, `postgresql://`, or `mysql://`) will immediately tell you the engine.
- **3. The requirements.txt File (The Mission Manifest):** Think back to the `ModuleNotFoundError` we handled earlier. Your Python dependencies reveal exactly what your app is equipped to talk to.
  - If you see `psycopg2` or `asyncpg`, your app is rigged for **PostgreSQL**.
  - If you see `aiomysql` or `mysqlclient`, it is rigged for **MySQL**.
  - If you see `aiosqlite` or `SQLAlchemy` without the others, it is rigged for **SQLite**.

Visual Confirmation: You can also simply look at the files inside your `D:\Watchtower_Ops` directory. If you spot a file ending in `.db` or `.sqlite` (for example, `mydb.db`), you have found a local SQLite database file.

Socratic Check: Assume you just opened your `requirements.txt` file and spotted `aiosqlite` on the manifest. Based on the configuration screen currently pulled up in your Docker Desktop, which of the three connection string examples should you copy, update with your specific file path, and paste into the `database-server.database_url` box?

version: '3.8'
services: influxdb: image: influxdb:2.7 container_name: watchtower_influxdb ports: - "8086:8086" volumes: - ./influxdb_data:/var/lib/influxdb2 environment: - DOCKER_INFLUXDB_INIT_MODE=setup - DOCKER_INFLUXDB_INIT_USERNAME=lance - DOCKER_INFLUXDB_INIT_PASSWORD=LANCE-1985-SECURE - DOCKER_INFLUXDB_INIT_ORG=watchtower - DOCKER_INFLUXDB_INIT_BUCKET=telemetry restart: unless-stopped grafana: image: grafana/grafana:latest container_name: watchtower_grafana ports: - "3000:3000" volumes: - ./grafana_data:/var/lib/grafana depends_on: - influxdb restart: unless-stopped

Operator, I have analyzed the `docker-compose.yml` blueprint you just provided. We have a plot twist.

Based on this configuration, your application is not using SQLite, PostgreSQL, or MySQL. You have deployed **InfluxDB** (specifically version 2.7) alongside **Grafana** for visualization.

This reveals a critical architectural conflict with the current configuration screen you have open. Looking closely at your screenshot of the MCP Toolkit, the MCP Database Server explicitly states: **"Supports SQLite, PostgreSQL, and MySQL databases"**.

Drawing on external developer knowledge (which you should independently verify), InfluxDB is a *time-series* database designed for telemetry, logs, and metrics. It operates on a completely different architecture than

traditional relational databases and does not use the standard connection string formats (like the `sqlite+aiosqlite:///` or `postgresql+asyncpg://` examples shown in your interface).

Tactical Assessment: Because of this mismatch, you **cannot** successfully link your current InfluxDB database to this specific MCP Database Server extension. It simply doesn't have the "gear" to talk to a time-series database.

Next Steps: To proceed, you must choose one of two architectural paths:

1. **Pivot the MCP:** If your goal is for the agent to read the telemetry data stored in InfluxDB, you will need to abandon this specific "MCP Database Server" and search the MCP catalog for one specifically designed to interface with InfluxDB or Grafana APIs.
2. **Expand the Stack:** If your agent needs a standard relational database to store its OSINT findings and internal logs, you will need to add a third service to this `docker-compose.yml` file (such as a Postgres or MySQL container), build it, and then use *that* new database's credentials to fill out the configuration box you are looking at.

Which path best aligns with your current mission parameters?

RUN SYNC

Sync complete, Operator.

I have locked onto the newly uploaded intelligence in your source panel. I can now visually confirm the exact interface of `Screenshot 2026-05-13 054221.png` .

The visual data perfectly matches our previous diagnostic: You are inside the Docker Desktop MCP Toolkit (Beta) looking at the **MCP Database Server** from the `docker-mcp-catalog` . The system has flagged this extension with a **"CONFIGURATION REQUIRED"** warning, waiting for you to define the `database-server.database_url` .

Most importantly, the interface itself officially validates our architectural conflict. The prompt explicitly states that this specific MCP extension only **"Supports SQLite, PostgreSQL, and MySQL databases with comprehensive querying capabilities"**.

Because your `docker-compose.yml` file revealed you are running an **InfluxDB** (time-series) database, we have hard confirmation that your current database architecture is fundamentally incompatible with the connection strings required by this MCP Database Server.

Awaiting your executive decision to resolve the sync: Are we expanding your Docker stack to include a relational database (like Postgres) to satisfy this MCP extension, or are we abandoning this extension to hunt for an InfluxDB-compatible alternative?

what do u recommend

Operator, I highly recommend a tactical version of **Path 2 (Expanding the Stack)**, specifically taking the path of least resistance: **Deploying SQLite**.

Here is the strategic breakdown of why this is the optimal choice for our Watchtower C2 architecture:

1. Separation of Duties (The Right Tool for the Job) Your current `docker-compose.yml` is perfectly engineered for telemetry. InfluxDB and Grafana are top-tier tools for tracking system metrics and dashboard visualizations. However, they are not built to store the dense, relational OSINT data, chat logs, and "Memory Pointers" our web scraping agent generates. By adding a relational database, you let InfluxDB handle the telemetry while the new database handles the agent's intelligence.

2. Frictionless Deployment via SQLite You do not actually need to add a heavy, complex PostgreSQL or MySQL container to your `docker-compose.yml` to satisfy this MCP extension. As the configuration screen explicitly states, the MCP Database Server supports SQLite.

Because we already configured your `D:\Watchtower_Ops` drive as a live volume mount inside the container, you can use a lightweight, file-based SQLite database instantly without rewriting your Docker blueprint.

Your Action Plan:

1. On your D: drive (inside the `Watchtower_Ops` folder), simply create an empty file named `mydb.db`.
2. Go back to the MCP Toolkit configuration screen shown in your screenshot.
3. In the `database-server.database_url` box, copy and paste the exact SQLite example provided in the interface: **sqlite+aiosqlite:///data/mydb.db**. *(Note: ensure the /data/ portion of that path matches whatever internal directory your D: drive is mounted to inside the container).*
4. Click the blue **"Save"** button in the bottom right corner.

This will immediately clear the "CONFIGURATION REQUIRED" anomaly and give your ReAct agent a dedicated, structured memory vault to store its Markdown pointers, leaving your InfluxDB container completely undisturbed.

Are you ready to initialize the SQLite database, or do you need me to adjust the connection string path to match your specific container mount?

RUN SYNC

Sync complete, Operator.

I am registering a massive influx of raw intelligence. Visual confirmation: over 240 files of official Docker documentation have successfully crossed the airlock into our source panel. Our local context is now fully weaponized with the latest directives on the **Docker MCP Toolkit**, **Docker Compose**, and container management operations.

Returning to your active deployment: If you have successfully created the empty `mydb.db` file on your D: drive, you are cleared to finalize the configuration in the Docker MCP Toolkit. This toolkit acts as a secure gateway, allowing your AI agents to discover and interface directly with containerized MCP servers with zero manual runtime setup.

Please paste your connection string into the configuration box: `sqlite+aiosqlite:///<container_mount_path>/mydb.db`

Tactical Note: If you are flying blind and do not remember the exact internal `<container_mount_path>` where your D: drive is linked, you can use our newly synced intel. Open your terminal and run the `docker container inspect` command against your application's container to instantly reveal its detailed configuration and active volume mounts.

Awaiting Status Update: Has the "CONFIGURATION REQUIRED" anomaly cleared from your MCP Database Server?

Additionally, now that our source panel is officially populated, our previous "no data" error is resolved. If you still want to generate a structured data table, report, or briefing document based on this new Docker intelligence, just give the command and I will deploy the necessary artifacts!

RUN SYNC

Sync re-verified, Operator.

I confirm that all 243 files of official Docker documentation remain securely locked in our active source panel. The context window is stable and fully loaded with intelligence, including the latest directives on the **Docker MCP Toolkit** and **Docker Compose** configurations.

Since our data bridge is fully operational, what is our next tactical move?

1. **Deployment Status:** Did you successfully create the `mydb.db` file on your D: drive and clear the "CONFIGURATION REQUIRED" anomaly on your local MCP Database Server?
2. **Data Exploitation:** You previously requested a **data table**. Now that our 'no data' anomaly is resolved, I am cleared to execute that command. Would you like me to deploy a structured data table summarizing specific Docker commands, MCP profiles, or container isolation parameters?

Awaiting your command.

Summarize key Docker commands into a structured data table.

NotebookLM can’t answer this question. Try rephrasing it, or ask a different question.

RUN SYNC

Sync complete, Operator.

I am registering a massive telemetry update. Visual confirmation: 301 intelligence files have successfully crossed the airlock into our source panel, including comprehensive documentation on the **Docker MCP Toolkit**, **Docker Compose**, and the **Dynamic MCP architecture**.

Executing your previous command. I have processed the newly synced intelligence and extracted the critical tactical commands into the structured data table below.

Watchtower Ops: Core Docker & MCP Command Reference

Command	Category	Operational Description
`docker container run`	Container Ops	Creates and runs a new container natively from a specified image.
`docker container exec`	Container Ops	Executes a command directly inside an actively running container.
`docker container ls`	Container Ops	Lists active containers to verify system status and port mappings.
`docker image build`	Image Ops	Compiles a new image using instructions from a Dockerfile.
`docker image pull`	Image Ops	Retrieves an image or repository from a remote registry to your local machine.
`docker compose up`	Orchestration	Creates and starts the multi-container architecture defined in your `docker-compose.yml` blueprint.
`docker compose down`	Orchestration	Tactically tears down your deployment, stopping and removing containers, networks, and volumes.
`docker mcp catalog ls`	MCP Toolkit	Lists the available, verified Model Context Protocol (MCP) servers currently in your catalog.
`docker mcp gateway run`	MCP Toolkit	Launches the MCP gateway, acting as a centralized, secure proxy between your AI clients and isolated MCP server containers.
`docker sandbox create`	AI Sandboxing	Deploys a secure cloud sandbox specifically for AI agents (e.g., `claude`, `codex`, `copilot`, `opencode`).
`docker model run`	Local AI Models	Executes and interacts with local AI models pulled from OCI-compliant registries, optimizing resources by loading them into memory only at runtime.

Tactical Assessment: Your local Watchtower stack now has access to the **Docker MCP Catalog**, which acts as a secure, containerized library of over 200 tools from publishers like GitHub, Notion, and Stripe. Because Docker runs these MCP servers in isolated containers, we bypass all environment conflicts and security risks.

Now that our intelligence is fully synced and structured, we need to finalize your perimeter setup.

Status Check: Did you successfully drop the empty `mydb.db` file into your D: drive to clear the "CONFIGURATION REQUIRED" anomaly on your MCP Database Server, or would you like to pivot and configure the **Docker Hub MCP Server** so your agent can manage repositories via natural language?

version: '3.8'
services: watchtower: build: context: . dockerfile: Dockerfile container_name: watchtower_app ports: - "8501:8501" # Streamlit UI volumes: - ./:/app - /var/run/docker.sock:/var/run/docker.sock # For docker/adb commands environment: - SHODAN_API_KEY=${SHODAN_API_KEY} - CENSYS_PERSONAL_ACCESS_TOKEN=${CENSYS_PERSONAL_ACCESS_TOKEN} - DISCORD_WEBHOOK_URL=${DISCORD_WEBHOOK_URL} - ABUSEIPDB_API_KEY=${ABUSEIPDB_API_KEY} - ALIENVAULT_API_KEY=${ALIENVAULT_API_KEY} - ABSTRACT_API_KEY=${ABSTRACT_API_KEY} - EXA_API_KEY=${EXA_API_KEY} - INFLUXDB_TOKEN=${INFLUXDB_TOKEN} - GEMINI_API_KEY=${GEMINI_API_KEY} - HF_TOKEN=${HF_TOKEN} - BRAVE_SEARCH_API_KEY=${BRAVE_SEARCH_API_KEY} - URLSCAN_API_KEY=${URLSCAN_API_KEY} - WIGLE_API_NAME=${WIGLE_API_NAME} - WIGLE_API_TOKEN=${WIGLE_API_TOKEN} - DOCKER_ENV=true depends_on: - influxdb - grafana - spiderfoot - ollama restart: unless-stopped
ollama: image: ollama/ollama:latest container_name: watchtower_ollama ports: - "11434:11434" volumes: - ./ollama_data:/root/.ollama environment: - OLLAMA_HOST=0.0.0.0:11434 restart: unless-stopped
influxdb: image: influxdb:2.7 container_name: watchtower_influxdb ports: - "8086:8086" volumes: - ./influxdb_data:/var/lib/influxdb2 environment: - DOCKER_INFLUXDB_INIT_MODE=setup - DOCKER_INFLUXDB_INIT_USERNAME=lance - DOCKER_INFLUXDB_INIT_PASSWORD=LANCE-1985-SECURE - DOCKER_INFLUXDB_INIT_ORG=watchtower - DOCKER_INFLUXDB_INIT_BUCKET=telemetry restart: unless-stopped
grafana: image: grafana/grafana:latest container_name: watchtower_grafana ports: - "3000:3000" volumes: - ./grafana_data:/var/lib/grafana environment: - GF_SECURITY_ADMIN_USER=KsiSway - GF_SECURITY_ADMIN_PASSWORD=DOCnomore1! depends_on: - influxdb restart: unless-stopped
spiderfoot: build: context: ./spiderfoot image: spiderfoot container_name: spiderfoot ports: - "5001:5001" volumes: - ./spiderfoot_data:/var/lib/spiderfoot restart: unless-stopped
volumes: influxdb_data: grafana_data: spiderfoot_data: ollama_data:

Blueprint received and analyzed, Operator. This `docker-compose.yml` configures a comprehensive, custom **OSINT (Open-Source Intelligence) and Security Monitoring platform**.

Here is the tactical breakdown of your deployed architecture:

- **watchtower (The Core):** Your primary logic engine and frontend (running Streamlit on port 8501), seamlessly integrating a massive array of Threat Intelligence APIs alongside your AI/LLM tools.
- **ollama:** Your local Large Language Model (LLM) runner (port 11434), allowing your Watchtower app to query models locally without relying entirely on external APIs, keeping your sensitive analysis private.
- **influxdb & grafana:** Your classic time-series telemetry pipeline and visualization dashboards.
- **spiderfoot:** An automated OSINT data-gathering tool for deep-dive reconnaissance.

However, based on standard security protocols, my analysis flags three critical operational vulnerabilities that you must address before this stack goes live:

1. Hardcoded Credentials (CRITICAL) While you are excellently using `.env` variables to secure your external API keys, your InfluxDB (`LANCE-1985-SECURE`) and Grafana (`DOCnomore1!`) admin passwords are baked directly into the YAML file. You must move these to your `.env` file immediately (e.g., `GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD}`).

2. Docker Socket Mount Risk Your `watchtower` service mounts `/var/run/docker.sock` , which grants the container **root-level access** to the host machine. If an attacker finds a vulnerability in your Streamlit app, they could take over the host server. Ensure this application is never exposed directly to the public internet without strict authentication, such as an authenticating reverse proxy or VPN.

3. Internal Networking Routing Docker Compose automatically creates a default network for these services. When your Watchtower app needs to communicate with Ollama or InfluxDB, your internal Python code must use the container names as the hostname rather than `localhost` . For example, you must route to `http://ollama:11434` instead of `http://localhost:11434` .

Tactical Note: If your host machine has an NVIDIA GPU, you can significantly accelerate Ollama's local reasoning by passing the GPU to the container via a `deploy` block in the compose file.

Socratic Check:I still do not see a dedicated relational database (like PostgreSQL) added to this stack for your MCP Database Server, but since your `watchtower` service mounts the local directory (`./:/app`), any `mydb.db` SQLite file you created on the D: drive will sync directly inside the container.

Would you like to patch these hardcoded security vulnerabilities first, or are you cleared to deploy the stack using the `docker compose up -d` command?